

11.

The car dataset captures the selling price of the used cars with respect to features like year_bought, km_driven, transmission and owner.

Objectives:

1. Understand the Dataset & cleanup (if required).
2. Build Regression model to predict the selling prices w.r.t year brought and km_driven
3. Also evaluate the model using scores RMSE

```
import pandas as pd
#Load Dataset
df = pd.read_csv('used_cars.csv')
print(df.head())
```

	year_bought	km_driven	transmission	owner	selling_price
0	2011.0	159736.0	Manual	Third	4928.821782
1	2019.0	72172.0	Automatic	Third	12699.765188
2	2015.0	98264.0	Automatic	First	9492.131132
3	2012.0	31736.0	Automatic	First	9914.594542
4	2011.0	117859.0	Manual	First	6767.338316

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 151 entries, 0 to 150
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   year_bought     150 non-null   object  
1   km_driven       150 non-null   float64 
2   transmission    151 non-null   object  
3   owner          151 non-null   object  
4   selling_price   150 non-null   float64 
dtypes: float64(2), object(3)
memory usage: 6.0+ KB
None
```

```
df.isnull().sum()
```

year_bought	1
km_driven	1
transmission	0
owner	0

```
selling_price    1
dtype: int64
```

Converting to Numeric Value

```
df['year_bought'] = pd.to_numeric(df['year_bought'], errors='coerce')
df['km_driven'] = pd.to_numeric(df['km_driven'], errors='coerce')
df['selling_price'] = pd.to_numeric(df['selling_price'],
errors='coerce')
```

```
df['year_bought'].fillna(df['year_bought'].mean(), inplace=True)
df['km_driven'].fillna(df['km_driven'].mean(), inplace=True)
df['selling_price'].fillna(df['selling_price'].mean(), inplace=True)
```

C:\Users\Anshul Parkar\AppData\Local\Temp\

ipykernel_12268\1053412413.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['year_bought'].fillna(df['year_bought'].mean(), inplace=True)
```

C:\Users\Anshul Parkar\AppData\Local\Temp\

ipykernel_12268\1053412413.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['km_driven'].fillna(df['km_driven'].mean(), inplace=True)
```

C:\Users\Anshul Parkar\AppData\Local\Temp\

ipykernel_12268\1053412413.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['selling_price'].fillna(df['selling_price'].mean(), inplace=True)
df.isnull().sum()
```

```
year_bought    0
km_driven      0
transmission    0
owner          0
selling_price  0
dtype: int64
```

```
# fix unrealistic values
```

```
df.loc[df['km_driven'] < 0, 'km_driven'] = df['km_driven'].mean()
```

```
# Replace invalid owner
```

```
df.loc[~df['owner'].isin(['First', 'Second', 'Third']), 'owner'] = 'Second'
```

```
# Remove duplicates
```

```
df.drop_duplicates(inplace=True)
```

```
print("Data cleaned successfully!\n")
```

```
print(df.head())
```

Data cleaned successfully!

	year_bought	km_driven	transmission	owner	selling_price
0	2011.0	159736.0	Manual	Third	4928.821782
1	2019.0	72172.0	Automatic	Third	12699.765188
2	2015.0	98264.0	Automatic	First	9492.131132
3	2012.0	31736.0	Automatic	First	9914.594542
4	2011.0	117859.0	Manual	First	6767.338316

```
import numpy as np
```

```
# Use 'year_bought' and 'km_driven' as features
```

```
x1 = df['year_bought'].values
```

```
x2 = df['km_driven'].values
```

```
y = df['selling_price'].values
```

```
# Feature scaling (normalization)
```

```
x1 = (x1 - np.mean(x1)) / np.std(x1)
```

```
x2 = (x2 - np.mean(x2)) / np.std(x2)
```

```
y = (y - np.mean(y)) / np.std(y)
```

```
# Combine features into matrix X (with bias term)
X = np.c_[np.ones(len(x1)), x1, x2] # shape: (n, 3)
y = y.reshape(-1, 1)
n = len(y)
```

1 Hypothesis Function (Model Definition)

The hypothesis (model) predicts y based on input x :

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

or in vectorized form (for multiple features):

$$h_{\theta}(X) = X \cdot \theta$$

```
def hypothesis(X, theta):
    return np.dot(X, theta)
```

2 Cost Function (Mean Squared Error)

We measure how well our model fits the data using the cost (or loss) function:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

```
def compute_cost(X, y, theta):
    m = len(y)
    predictions = X.dot(theta)
    cost = (1/(2*m)) * np.sum((predictions - y)**2)
    return cost
# This gives a single number - smaller = better fit.
```

3 Compute Gradient (Partial Derivatives)

To minimize cost, we find the direction of steepest descent using partial derivatives:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} X^T (h_{\theta}(X) - y)$$

```
def compute_gradient(X, y, theta):
    m = len(y)
    predictions = X.dot(theta)
    gradient = (1/m) * X.T.dot(predictions - y)
    return gradient
# This gives how much each parameter ( $\theta_0, \theta_1, \dots$ ) should change to
# reduce cost.
```

4 Gradient Descent Algorithm (Parameter Update Rule)

We iteratively adjust the parameters:

$$\theta := \theta - \alpha \times \nabla_{\theta} J(\theta)$$

where:

- α = learning rate (step size)
- $\nabla_{\theta} J(\theta)$ = gradient (direction to move)

```
def gradient_descent(X, y, theta, alpha, iterations):
    cost_hist = []
    for i in range(iterations):
        gradient = compute_gradient(X, y, theta)
        theta = theta - alpha * gradient
        cost = compute_cost(X, y, theta)
        cost_hist.append(cost)
    return theta, cost_hist

# 7 TRAIN MODEL
# =====
# initialize theta (3 x 1) => [theta0 (bias), theta_year, theta_km]
theta_init = np.zeros((X.shape[1], 1))
alpha = 0.05          # learning rate - tune if needed
iters = 2000          # increase/decrease for better convergence

theta_final, cost_hist = gradient_descent(X, y, theta_init,
alpha=alpha, iterations=iters)

print("\n Training finished.")
print("Learned parameters (normalized scale):")
print(theta_final.ravel())
```

Training finished.

```

Learned parameters (normalized scale):
[-1.83168295e-15  7.87657331e-01 -3.85966926e-01]

print("\n Training Complete!")
print(f"Final Weights ( $\theta_0$ ,  $\theta_1$ ,  $\theta_2$ ): {theta_final.ravel()}")
print(f"Final Cost (MSE): {cost_hist[-1]:.6f}")

# -----
# Step 6: Evaluate Model
# -----
y_pred = X.dot(theta_final)
MSE = (1/n) * np.sum((y_pred - y)**2)
RMSE = np.sqrt(MSE)

print(f"\nModel Evaluation:")
print(f"MSE = {MSE:.6f}")
print(f"RMSE = {RMSE:.6f}")

 Training Complete!
Final Weights ( $\theta_0$ ,  $\theta_1$ ,  $\theta_2$ ): [-1.83168295e-15  7.87657331e-01 -
3.85966926e-01]
Final Cost (MSE): 0.119206

Model Evaluation:
MSE = 0.238413
RMSE = 0.488275

# Step 3: Define Cost and Gradient Functions
# -----
def mse_cost(w):
    y_hat = X.dot(w)
    return (1/n) * np.sum((y_hat - y)**2)

def mse_grad(w):
    y_hat = X.dot(w)
    err = (y_hat - y)
    grad = (2/n) * X.T.dot(err)
    return grad

# -----
# Step 4: Gradient Descent Implementation
# -----
def gradient_descent(f, grad, w_init, alpha=0.05, iters=500):
    w = w_init.copy().astype(float)
    cost_history = []
    for i in range(iters):
        w -= alpha * grad(w)
        cost_history.append(f(w))
    return w, cost_history

```

```

# Initialize parameters ( $\theta_0$ ,  $\theta_1$ ,  $\theta_2$ )
w_init = np.zeros((3,1))

# Run gradient descent
final_w, cost_history = gradient_descent(mse_cost, mse_grad, w_init,
alpha=0.1, iters=1000)

# -----
# Step 5: Display Results
# -----
print("\n Training Complete!")
print(f"Final Weights ( $\theta_0$ ,  $\theta_1$ ,  $\theta_2$ ): {final_w.ravel()}")
print(f"Final Cost (MSE): {cost_history[-1]:.6f}")

# -----
# Step 6: Evaluate Model
# -----
y_pred = X.dot(final_w)
MSE = (1/n) * np.sum((y_pred - y)**2)
RMSE = np.sqrt(MSE)

print(f"\nModel Evaluation:")
print(f"MSE = {MSE:.6f}")
print(f"RMSE = {RMSE:.6f}")

 Training Complete!
Final Weights ( $\theta_0$ ,  $\theta_1$ ,  $\theta_2$ ): [-1.83260814e-15  7.87657331e-01 -
3.85966926e-01]
Final Cost (MSE): 0.238413

Model Evaluation:
MSE = 0.238413
RMSE = 0.488275

```